Ejercicio · UD02 · Bloque 2

Diseña la interacción de un asistente multimodal

Asignatura	Desarrollo de Interfaces · 2.º DAM
Unidad	UD02 · Generación de interfaces naturales de usuario
Contenido trabajado	Diapositivas 19–26 · Bloque 2: Interfaces naturales, tipos (PDF, apartado 2, pp. 14–18)
Temas	Canales y tipos de NUI · entrada explícita frente a interpretada · estados (NaturalUiState) · evolución de la interacción · eventos GUI/NUI en JavaFX · tipos de interfaz y cuándo usarlos · campos de aplicación · política de interacción (InteractionPolicy)
Modalidad	Individual

Contexto

Un taller mecánico quiere una aplicación para tableta llamada TallerGuía. La tableta va sujeta a un soporte junto al vehículo y el mecánico la usa mientras trabaja, normalmente con guantes y las manos sucias. El taller es ruidoso (compresores, herramientas neumáticas, radio).

La aplicación debe permitir:

- Avanzar, repetir o volver al paso anterior de una reparación guiada.
- Señalar sobre la imagen de la cámara la pieza a la que se refiere el paso (realidad aumentada).
- Pedir un recambio al almacén (genera un pedido con coste para el taller).
- Cerrar la orden de trabajo (el vehículo pasa a «listo para entregar» y se avisa al cliente).
- Consultar el manual técnico del vehículo.

Las intenciones internas que usa la aplicación son: NEXT_STEP, REPEAT_STEP, SHOW_MANUAL, MARK_PART, ORDER_PART y CLOSE_WORK_ORDER.

Ejercicio

a) Canales de interacción (diap. 19 y 25)

- **Indica qué canales de interacción combinarías en TallerGuía y cuál sería el canal dominante. ¿Qué tipo de NUI es según su señal?**

  En TallerGuía combinaría voz, táctil, gestual/corporal y AR (cámara + superposición). El canal dominante sería la voz, porque el mecánico trabaja con guantes, las manos sucias y a menudo ocupadas; la voz es la única entrada que no exige contacto ni precisión motora fina. El táctil y el gestual quedan como canales de apoyo para acciones que requieren precisión o confirmación; la AR es un canal de salida (señalización visual sobre la pieza), no de entrada principal.

  Según su señal, es una NUI multimodal: combina entrada explícita (pulsar el botón del micrófono, tocar la pantalla) con entrada interpretada (reconocimiento de voz, gestos, detección de la pieza sobre la imagen). No es una interfaz tradicional porque el usuario no depende de teclado/ratón y porque el sistema interpreta lenguaje natural y señales del entorno.

- Asigna un canal a cada una de las cinco funciones de la lista y justifica la elección. Ten en cuenta las condiciones del taller.

Función	Canal	Justificación
Avanzar / repetir / volver al paso anterior	**Voz** (principal) + **gestual** (apoyo)	Manos ocupadas/sucias; gestos amplios (p. ej. deslizar) como respaldo si hay ruido.
Señalar sobre la imagen la pieza (AR)	**Táctil + AR**	Requiere precisión espacial sobre la imagen; la voz no puede indicar *dónde* está la pieza. La AR superpone la señalización.

Cerrar la orden de trabajo	**Voz** (con confirmación)	Es una acción con coste económico; la voz agiliza, pero debe confirmarse por el riesgo.
Cerrar la orden de trabajo	**Voz + táctil** (confirmación)	Acción crítica e irreversible (avisa al cliente); exige confirmación explícita.
Consultar el manual técnico	**Voz** (consulta) + **táctil** (navegación)	La voz abre el manual; el táctil permite hojear, hacer zoom o buscar.

- **Según su campo de aplicación, ¿qué requisitos de precisión, trazabilidad y validación debería cumplir? Compáralo con una app de ocio.**

  Precisión: alta en MARK_PART (la pieza debe señalarse sin ambigüedad) y en ORDER_PART (referencia exacta del recambio). La voz debe tolerar ruido del taller.

  Trazabilidad: cada acción debe quedar registrada (quién, cuándo, qué intención, con qué confianza). Especialmente ORDER_PART y CLOSE_WORK_ORDER, que generan coste y aviso al cliente.

  Validación: ORDER_PART y CLOSE_WORK_ORDER requieren confirmación explícita; NEXT_STEP, REPEAT_STEP, SHOW_MANUAL pueden ejecutarse con manos libres.

  Comparación con una app de ocio: en ocio la precisión puede ser baja (si falla, se repite), la trazabilidad no es crítica y no hay validación obligatoria. En TallerGuía, un error en ORDER_PART cuesta dinero y un error en CLOSE_WORK_ORDER entrega un vehículo mal reparado. Por eso el coste del error es mucho mayor y la política de interacción debe ser más estricta.

b) Entrada interpretada y estados (diap. 20 y 21)

El mecánico pulsa el botón del micrófono y dice: «pide un filtro de aceite».

- **Escribe la secuencia de estados de NaturalUiState por la que pasa la interfaz hasta que el pedido se realiza, y describe qué ve el mecánico en la pantalla en cada estado.**

  IDLE → Pantalla en reposo, mostrando el paso actual de la reparación. El botón del micrófono está disponible.

  LISTENING → El botón del micrófono se deshabilita y se muestra «Escuchando…». Se captura el audio.

  PROCESSING → Se muestra un indicador de carga («Interpretando…»). El sistema convierte voz a texto y clasifica la intención.

  CONFIRMING → Se muestra la interpretación: «¿Pedir filtro de aceite? [Confirmar] [Cancelar]». El sistema espera confirmación porque ORDER_PART tiene coste.

  EXECUTING → Tras confirmar, se muestra «Enviando pedido al almacén…». Se ejecuta la acción.

  IDLE (vuelta) → Se muestra «Pedido realizado: filtro de aceite» y se vuelve al reposo.

- **Indica dos cosas que pueden fallar en la interpretación de esta orden en el taller y en qué estado acabaría la interfaz en cada caso.**

  Ruido del taller (compresores, radio): el reconocedor no entiende bien la frase o la confunde con otra intención. La interfaz acabaría en ERROR (no se pudo interpretar) o en CONFIRMING con una interpretación errónea que el mecánico debe corregir.

  Ambigüedad del lenguaje («pide un filtro» sin especificar tipo): el sistema no sabe qué filtro. Acabaría en CONFIRMING pidiendo aclaración, o en ERROR si no hay candidatos claros.

- **Explica cómo sale la interfaz del estado ERROR sin bloquear al mecánico.**
  El estado ERROR no debe ser un callejón sin salida. Opciones:

  Mostrar un mensaje claro: «No te he entendido. ¿Quieres pedir un filtro de aceite?».

  Ofrecer reintentar (vuelve a LISTENING) o cancelar (vuelve a IDLE).

  Permitir corregir por escrito o táctil la intención detectada.

  Tras un tiempo sin respuesta, volver automáticamente a IDLE para no dejar la pantalla bloqueada.

  La clave es que el mecánico siempre tenga una salida a un toque (o a una palabra) y que el sistema no quede atrapado esperando una confirmación que no llega.

c) ¿NUI o interfaz tradicional? (diap. 22 y 24)

Para cada tarea, indica el tipo de interfaz más adecuado (tradicional, voz, gestual/corporal o inmersiva AR/VR) y justifícalo con el criterio «cuándo usarla»:

Tarea	Tipo de interfaz	Justificación
1. Avanzar al siguiente paso de la reparación	**Voz** (NUI)	Manos ocupadas/sucias; la acción es frecuente y de bajo riesgo.
2. Ver dónde está el sensor dentro del motor	**Inmersiva AR**	Requiere superponer información sobre la imagen real del motor para localizar la pieza.
3. Emitir la factura al cliente	Tradicional	Acción administrativa, con datos fiscales, importes y validación; requiere precisión y trazabilidad.
4. Corregir el stock de 40 referencias en el inventario	Tradicional	Edición masiva de datos; el teclado/formulario es más rápido y preciso que la voz.

- Con la idea de la diapositiva 22, explica qué significa que «la complejidad no desaparece, se traslada» en el caso de TallerGuía.
  Significa que al pasar de una interfaz tradicional a una NUI no eliminamos la complejidad del sistema: la **movemos** del usuario al diseñador y al motor de interpretación. En TallerGuía, el mecánico ya no tiene que buscar botones con las manos sucias, pero **alguien** tiene que:
  - Entrenar y mantener el modelo de voz para el ruido del taller.
  - Definir la política de confirmación para acciones con coste.
  - Gestionar los estados (LISTENING, PROCESSING, CONFIRMING, ERROR…).
  - Resolver ambigüedades y errores de interpretación.

  Es decir, la carga se traslada del **usuario** (menos esfuerzo motor) al **sistema** (más lógica, más estados, más validación). La complejidad sigue existiendo, solo cambia de sitio

d) Eventos en JavaFX (diap. 23)

- Escribe tres eventos de tipo NUI que generaría TallerGuía y otros tres de tipo GUI clásica.
  Tres eventos NUI y tres eventos GUI
  NUI:

1. vozDetectada → se ha reconocido una frase del mecánico.

2. gestoDeslizar → el mecánico desliza la mano para avanzar de paso.
3. piezaSenalada → la cámara detecta que el mecánico señala una pieza.

GUI clásica:

1. botonSiguientePulsado → clic en el botón «Siguiente paso».
2. menuManualSeleccionado → selección de una opción del menú del manual.
3. campoTextoModificado → el mecánico escribe una referencia de recambio.

- Parte del ejemplo de la diapositiva 23 y amplíalo para que la pantalla guarde su estado con NaturalUiState:
  - un atributo estado que empieza en IDLE;
  - un método cambiarEstado(NaturalUiState nuevo, String texto) que actualiza el atributo y la etiqueta;
  - el botón «Activar modo voz» pasa a LISTENING y queda deshabilitado mientras escucha;
  - un botón «Cancelar» que vuelve a IDLE y solo está habilitado cuando no se está en reposo.

```
Button voiceButton = new Button("Activar modo voz");
Label statusLabel = new Label("Sistema en espera");

voiceButton.setOnAction(event -> {
    statusLabel.setText("Escuchando comando del usuario...");
    // Aquí se conectaría el servicio de reconocimiento de voz.
});
```

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class TallerGuiaApp extends Application {

   // Estado inicial
   private NaturalUiState estado = NaturalUiState.IDLE;

   private Label statusLabel;
   private Button voiceButton;
   private Button cancelButton;

   // Método que actualiza el estado y la etiqueta
   private void cambiarEstado(NaturalUiState nuevo, String texto) {
      this.estado = nuevo;
      statusLabel.setText(texto);

      // El botón de voz solo está habilitado en IDLE
      voiceButton.setDisable(nuevo != NaturalUiState.IDLE);

      // El botón Cancelar solo está habilitado cuando NO estamos en IDLE
      cancelButton.setDisable(nuevo == NaturalUiState.IDLE);
   }
```

```
    @Override
    public void start(Stage stage) {
        statusLabel = new Label("Sistema en espera");
        voiceButton = new Button("Activar modo voz");
        cancelButton = new Button("Cancelar");

        // Estado inicial
        cambiarEstado(NaturalUiState.IDLE, "Sistema en espera");

        // Botón de voz: pasa a LISTENING y se deshabilita
        voiceButton.setOnAction(event -> {
            cambiarEstado(NaturalUiState.LISTENING, "Escuchando comando del usuario...");
            // Aquí se conectaría el servicio de reconocimiento de voz.
        });

        // Botón Cancelar: vuelve a IDLE
        cancelButton.setOnAction(event -> {
            cambiarEstado(NaturalUiState.IDLE, "Sistema en espera");
        });

        VBox root = new VBox(10, statusLabel, voiceButton, cancelButton);
        Scene scene = new Scene(root, 320, 200);
        stage.setTitle("TallerGuía");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

e) Política de interacción (diap. 26)

Partimos de la clase InteractionPolicy de la unidad:

```
public class InteractionPolicy {
    public boolean requiresConfirmation(String intent) {
        return intent.equals("DELETE_DATA") || intent.equals("MAKE_PAYMENT");
    }

    public boolean canRunHandsFree(String intent) {
        return !requiresConfirmation(intent);
    }
}
```

- Completa la tabla sin ejecutar el código (si una llamada provoca un error, indícalo):

intent	requiresConfirmation	canRunHandsFree
"NEXT_STEP"	False	True

"MAKE_PAYMENT"	True	False
"ORDER_PART"	False	True
"UNKNOWN"	False	True
"delete_data"	False (“no coincide con delete_data”)	True
null	**Error:** NullPointerException	**Error:** NullPointerException

- A partir de la tabla, identifica al menos tres problemas que tendría usar esta clase tal cual en TallerGuía.

  No contempla las intenciones de TallerGuía. ORDER_PART y CLOSE_WORK_ORDER son acciones con coste o irreversibles, pero la clase solo confirma DELETE_DATA y MAKE_PAYMENT. Un pedido de recambio o el cierre de una orden se ejecutarían sin confirmación.

  Es sensible a mayúsculas/minúsculas. "delete_data" no coincide con "DELETE_DATA", así que una intención mal normalizada se ejecutaría sin confirmación. Es un fallo de robustez grave.

  No maneja null ni entradas desconocidas. null provoca NullPointerException y "UNKNOWN" se trata como acción segura, lo que puede ser peligroso si el sistema no ha entendido bien la orden.

- Escribe una clase TallerPolicy que resuelva esos problemas. ¿Qué intenciones de TallerGuía deben pedir confirmación y cuáles pueden ejecutarse con manos libres? Justifícalo.

```
import java.util.Set;

public class TallerPolicy {

    // Intenciones que requieren confirmación explícita
    private static final Set<String> REQUIREN_CONFIRMACION = Set.of(
        "ORDER_PART",        // genera un pedido con coste para el taller
        "CLOSE_WORK_ORDER"   // acción irreversible: avisa al cliente
    );

    // Intenciones que pueden ejecutarse con manos libres
    private static final Set<String> MANOS_LIBRES = Set.of(
        "NEXT_STEP",
        "REPEAT_STEP",
        "SHOW_MANUAL",
        "MARK_PART"
    );

    /**
     * Normaliza la intención: null y entradas desconocidas se tratan como UNKNOWN.
     */
    private String normalizar(String intent) {
        if (intent == null || intent.isBlank()) {
            return "UNKNOWN";
        }
        return intent.trim().toUpperCase();
    }
```

```
    /**
     * ¿Requiere confirmación explícita?
     * Por defecto, todo lo que no sea una intención segura conocida requiere confirmación.
     */
    public boolean requiresConfirmation(String intent) {
        String normalizada = normalizar(intent);

        if (REQUIREN_CONFIRMACION.contains(normalizada)) {
            return true;
        }
        // Si es una intención de manos libres conocida, no requiere confirmación
        if (MANOS_LIBRES.contains(normalizada)) {
            return false;
        }
        // Cualquier otra cosa (UNKNOWN, desconocida, etc.) requiere confirmación por seguridad
        return true;
    }

    /**
     * ¿Puede ejecutarse sin tocar la pantalla?
     * Solo las intenciones seguras conocidas.
     */
    public boolean canRunHandsFree(String intent) {
        String normalizada = normalizar(intent);
        return MANOS_LIBRES.contains(normalizada) && !requiresConfirmation(normalizada);
    }
}
```

Justificación de la política

- ORDER_PART → requiere confirmación porque **genera un pedido con coste** para el taller. Un falso positivo cuesta dinero.
- CLOSE_WORK_ORDER → requiere confirmación porque es **irreversible**: el vehículo pasa a «listo para entregar» y se avisa al cliente. Un error aquí tiene consecuencias graves.
- NEXT_STEP**,** REPEAT_STEP**,** SHOW_MANUAL → pueden ejecutarse con manos libres porque son **reversibles y de bajo riesgo**. Si el sistema se equivoca, el mecánico repite o corrige sin coste.
- MARK_PART → puede ejecutarse con manos libres porque es una acción **visual** (señalar una pieza), no genera coste ni cambia el estado del vehículo.
- UNKNOWN **o** null → por defecto **requieren confirmación**. Es más seguro preguntar que ejecutar algo que no se ha entendido.

Esta política resuelve los tres problemas: contempla las intenciones reales de TallerGuía, normaliza mayúsculas/minúsculas y trata null/desconocidos como casos que requieren confirmación.

Entrega

- Un documento con las respuestas de los apartados a) a e). Las respuestas deben estar razonadas: no basta con la solución, explica por qué.
- El código de los apartados d) y e) en archivos .java.
- Puedes usar la presentación y el PDF de la unidad. Cita la diapositiva o el concepto en el que te apoyas.